New Features of J2SE 5.0

Dr. Stephan Fischli

Fall 2004

Contents

Introduction

History of the Java Language

1995	JDK 1.0 First Public Release
1997	JDK 1.1 Inner Classes (function objects)
2001	JDK 1.4 Assertions (code verification)
2004	JDK 1.5 Generics et. al. (ease of development)

Naming and Versioning

Platform Names

- Java 2 Platform Standard Edition 5.0
- J2SE 5.0
- Tiger (code name)

Platform Products

- Java Developer's Kit JDK 1.5
- Java Runtime Environment JRE 1.5

New Features and Enhancements

- Java Language
- Base Libraries
- Integration Libraries
- User Interface
- Deployment
- Tools and Tool Architecture
- Virtual Machine
- Performance
- OS and Hardware Support

Compatibility

- JDK 1.5 is upwards source and binary compatible with Java 2 SDK 1.4.2 (except for some incompatibilities)
- Downward source or binary compatibility is not supported
- Deprecated APIs are available only for backwards compatibility but should not be used anymore

Compiler options

- source restricts the version of source code
- target determines the version of VM on which the generated code will work

References

- JDK 1.5 Download
  http://java.sun.com/j2se/1.5.0/download.jsp
- JDK 1.5 Documentation
  http://java.sun.com/j2se/1.5.0/docs/index.html
- Generics Tutorial
  http://java.sun.com/j2se/1.5/pdf/generics-tutorial.pdf
- Java 1.5 Tiger
  Brett McLaughlin, O'Reilly, 2004
- Java in a Nutshell
  David Flanagan, O'Reilly, 2005
- New Features in J2SE 5.0
  Angelika Langer, Seminar

Generics

Introduction

Generics are an enhancement of the Java type system:

- allow to use collections with compile-time type safety
- use type parameters instead of the unspecific type java.lang.Object
- move part of the specification from comments to signatures
- improve the readability and robustness of programs
- are implemented by type erasure and translation

Using Collections in Pre-Tiger

```
List list = new LinkedList();
list.add("foo");
list.add(new Integer(7));
...
for (int i = 0; i < list.size(); i++) {
   String s = (String)list.get(i);          // runtime error
   ...
}
```

- Elements of different types can be added to a collection
- Type casts are needed when retrieving an element

Using Generics

```
List<String> list = new LinkedList<String>();
list.add("foo");
list.add(new Integer(7));            // compile-time error
...
for (int i = 0; i < list.size(); i++) {
   String s = list.get(i);
   ...
}
```

- With a parameterized collection, the type of elements is specified by a type parameter
- The compiler guarantees that no other object types can be added
- No type casts are needed when retrieving elements

Iterating over Generics

```
List<String> list = new LinkedList<String>();
list.add("foo");
...
for (Iterator<String> i = list.iterator(); i.hasNext(); ) {
   String s = i.next();
   ...
}
```

- Iterators of a parameterized collection are parameterized with the same type parameter

Generics as Method Parameters

```
List<String> list = new LinkedList<String>();
list.add("foo");
...
list = toUpperCase(list);

public List<String> toUpperCase(List<String> list) {
   List<String> ucList = new LinkedList<String>();
   for (Iterator<String> i = list.iterator(); i.hasNext(); ) {
      String s = i.next();
      ucList.add(s.toUpperCase());
   }
   return ucList;
}
```

- Parameterized collections can be used as arguments and return types of methods

Implementing Generics

```
class Node<E> {
   E elem; Node<E> next;
   Node(E elem) { this.elem = elem; }
}
public class LinkedList<E> {
   private Node<E> head, tail;
   public void add(E elem) {
      Node<E> node = new Node<E>(elem);
      if (head == null) head = tail = node;
      else tail = tail.next = node;
   }
   ...
}
```

- A parameterized collection is implemented by using a formal type parameter

Generics and Raw Types

```
List list = new LinkedList();
list.add(new Integer(7));          // compile-time warning
...
List<Integer> intList = list;      // compile-time warning
```

- For backward compatibility unparameterized collections (raw types) can still be used
- When adding elements to a raw type or assigning a raw type to a parameterized type, the compiler generates unchecked warnings

Generics and Inheritance

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
List<Object> list = intList;          // compile-time error
Object o = list.get(0);
list.add("foo");
```

- There is no type relationship between the instantiations of a parameterized collection with different type parameters

Wildcard Types

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
List<?> list = intList;
Object o = list.get(0);
list.add("foo");                    // compile-time error
```

- A wildcard type denotes a collection whose elements are of unknown type
- Any parameterized type is assignment compatible to the corresponding wildcard type
- Adding elements to a wildcard type is not allowed

Wildcard Types as Method Parameters

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
println(intList);

public void println(List<?> list) {
   for (Iterator<?> i = list.iterator(); i.hasNext(); ) {
      Object o = i.next();
      System.out.print(o.toString() + " ");
   }
   System.out.println();
}
```

- Wildcard types can be used to implement methods that accept a parameterized collection with any type parameter

Bounded Wildcard Types

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
double sum = sum(intList);

public double sum(List<? extends Number> list) {
   double sum = 0;
   for (Iterator<? extends Number> i = list.iterator(); i.hasNext(); ) {
      Number n = i.next();
      sum += n.doubleValue();
   }
   return sum;
}
```

- A bounded wildcard type denotes a collection whose elements have a common super type
- Adding elements to a bounded wildcard is not allowed

Generic Methods

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
reverse(intList);

public <E> void reverse(List<E> list) {
   for (int i = 0; i < list.size() / 2; i++) {
      int j = list.size() - 1 - i;
      E e = list.get(i); list.set(i, list.get(j)); list.set(j, e);
   }
}
```

- Methods can be parameterized by introducing a formal type parameter

Method Parameters with Type Dependencies

```
List<Integer> intList = new LinkedList<Integer>();
intList.add(new Integer(7));
...
List<Number> newList = new LinkedList<Number>();
copy(newList, intList);

public <T,S extends T> void copy(List<T> dest, List<S> src) {
   for (Iterator<S> i = src.iterator(); i.hasNext(); ) {
      S s = i.next();
      dest.add(s);
   }
}
```

- Type parameters allow to express dependencies among the argument types of a generic method

Type Erasure and Translation

Generics are implemented by type erasure and translation:

1. All generic type information is erased
   e.g. `List<E>` is converted to `List`
2. Remaining type variables are replaced by the upper bound types
   e.g. `T x = ...` is translated to `Object x = ...`
3. Wherever necessary type casts are inserted
   e.g. `Number n = i.next()` is translated to `Number n = (Number)i.next()`

Caveats

- Primitive values are not allowed as type parameters
- Generic type information is not available at runtime
- Static fields and methods are shared among all instances
- No objects and arrays represented by a type parameter can be created

For-Each Loop

Iterating in Pre-Tiger

```
Integer[] numArray = new Integer[...];
List numList = new LinkedList();
...
int sum = 0;
for (int i = 0; i < numArray.length; i++) {
   Integer n = numArray[i];
   sum += n.intValue();
}
for (Iterator i = numList.iterator(); i.hasNext(); ) {
   Integer n = (Integer)i.next();
   sum += n.intValue();
}
```

- The index and the iterator are only used to access the elements of the array or the collection, respectively

Using the For-Each Loop

```
Integer[] numbers = new Integer[...];
List<Integer> numbers = new LinkedList<Integer>();
...
int sum = 0;
for (Integer n : numbers)
   sum += n.intValue();
```

The for-each loop

- allows to iterate over an array or a collection without using an index or an iterator
- is less error-prone
- combines nicely with generics

Loop Syntax

for (*declaration* : *expression*)
    statement

- *expression* must be either an array or an object that implements the interface java.lang.Iterable (which guarantees the existence of an iterator)
- *declaration* declares the loop variable to which the elements of the array or the iterable object are assigned
- *statement* contains the code that is executed for each element of the array or the iterable object

Caveats

The for-each loop cannot be used

- when the position of an element is needed
- for removing or replacing elements in a collection
- for loops that must iterate over multiple collections in parallel

Autoboxing and Unboxing

Wrapper Classes in Pre-Tiger

```
List numbers = new LinkedList();
numbers.add(new Integer(7));
...
Integer i = (Integer)number.get(0);
int n = i.intValue();
```

- When using primitive values with collections, the values must explicitly be converted to objects and vice versa

Using Autoboxing and Unboxing

```
List<Integer> numbers = new LinkedList<Integer>();
numbers.add(7);
...
int n = numbers.get(0);
```

- Autoboxing and unboxing provide automatic conversions between primitive values and their corresponding wrapper types

Calculating with Wrapper Types

```
List<String> words = new LinkedList<String>();
...
Map<String,Integer> frequency = new HashMap<String,Integer>();
for (String word : words) {
   Integer n = frequency.get(word);
   if (n == null) n = 1;
   else n++;
   frequency.put(word, n);
}
```

- Unboxing also enables to apply arithmetic and comparison operators to the wrapper types

Caveats

- Unboxing an object type whose value is null results in a java.lang.NullPointerException
- The == operator still compares the identity of references when applied to object types
- The resolution of overloaded methods may become more difficult
- Autoboxing and unboxing increases the performance costs

Enumeration Types (Enums)

Enumerated Values in Pre-Tiger

```
public class Heating {
   public static final int WINTER = 0;
   public static final int SPRING = 1;
   public static final int SUMMER = 2;
   public static final int FALL = 3;

   int season = ...
   if (season == SPRING) ...
}
```

Disadvantages:

- Not type safe (just integers)
- Not robust (constants are compiled into client code)

Defining Enums

```
public enum Season {
   WINTER, SPRING, SUMMER, FALL
}

Season season = ...
if (season == Season.SPRING) ...
```

Enums are full-fledged classes that

- are type safe
- are robust
- define a namespace
- have informative printed values
- can have fields and methods
- may implement arbitrary interfaces
- are comparable and serializable

Iterating over Enums

```
public enum Season { WINTER, SPRING, SUMMER, FALL }

for (Season season : Season.values()) {
   String s = season.toString();
   ...
}
```

- The static method values returns all the values of an enumeration
- The method toString returns the name of an enumeration value

Switching on Enums

```
public enum Coin { PENNY, NICKEL, DIME, QUARTER }

Coin coin = ...
int value;
switch (coin) {
   case PENNY : value = 1; break;
   case NICKEL : value = 5; break;
   case DIME : value = 10; break;
   case QUARTER : value = 25; break;
   default : throw new AssertionError("Unknown coin");
}
```

- Enumerations can be used in switch statements
- The values must not be qualified with the enumeration type

Enums with Fields and Methods

```
public enum Coin {
   PENNY(1), NICKEL(5), DIME(10), QUARTER(25);
   private final int value;
   private Coin(int value) { this.value = value; }
   public int value() { return value; }
}

Coin coin = ...
int value = coin.value();
```

- Enumerations can have fields and methods
- The fields must be initialized by a private constructor

Methods with Value-Specific Behavior

```
public enum Operation {
   PLUS, MINUS, TIMES, DIVIDE;
   public double eval(double x, double y) {
      switch (this) {
         case PLUS : return x + y;
         case MINUS : return x - y;
         case TIMES : return x * y;
         case DIVIDE : return x / y;
      }
      throw new AssertionError("Unknown operation");
   }
}

Operation op = ...
double result = op.eval(x, y);
```

- Value-specific behavior of an enumeration can be implemented by switching on the value

Value-Specific Methods

```
public enum Operation {
   PLUS { double eval(double x, double y) { return x + y; } },
   MINUS { double eval(double x, double y) { return x - y; } },
   TIMES { double eval(double x, double y) { return x * y; } },
   DIVIDE { double eval(double x, double y) { return x / y; } };
   abstract double eval(double x, double y);
}

Operation op = ...
double result = op.eval(x, y);
```

- Value-specific behavior can also be implemented by declaring an abstract method that is overridden in each value

Variable-Length Argument Lists (Varargs)

Variable Number of Arguments in Pre-Tiger

```
public double max(double[] values) {
   if (values.length == 0)
      throw new IllegalArgumentException("Missing values");
   double max = values[0];
   for (int i = 1; i < values.length; i++)
      if (values[i] > max) max = values[i];
   return max;
}

double[] values = { x, y, z };
double max = max(values);
```

- A method that takes a variable number of arguments must be implemented using an array parameter

Using Varargs

```
public double max(double... values) {
   if (values.length == 0)
      throw new IllegalArgumentException("Missing values");
   double max = values[0];
   for (double value : values)
      if (value > max) max = value;
   return max;
}

double max = max(x, y, z);
double max = max(new double[] { x, y, z });
double max = max();
```

- A vararg parameter allows the arguments to be passed as an array or a sequence
- Within the method the arguments are treated like an array

Static Imports

Using Static Members in Pre-Tiger

```
package org.iso;
public class Physics {
    public static final double GRAVITATIONAL_CONSTANT = 6.67259e-11;
    ...
}

import org.iso.Physics;
...
double energy = mass * Math.pow(Physics.GRAVITATIONAL_CONSTANT, 2);
```

- Static members must be qualified with the class they belong to

Importing Static Members

```
package org.iso;
public class Physics {
   public static final double GRAVITATIONAL_CONSTANT = 6.67259e-11;
   ...
}

import static java.lang.Math.pow;
import static org.iso.Physics.*;
...
double energy = mass * pow(GRAVITATIONAL_CONSTANT, 2);
```

- Static imports make the static members of a class available without qualification
- Static imports are especially useful for importing the values of an enumeration

Annotations

Introduction

Annotations allow to add metadata to a Java program:

- are assigned to program elements like modifiers
- take the form of name-value pairs
- are checked by the Java compiler
- can be used by tools (e.g. apt) to generate additional code

Standard Annotations

```
@Override public void run() { ... }
@Deprecated public void foo() { ... }
@SuppressWarnings("unchecked") public void bar() { ... }
```

Java defines three standard annotations:

- @Override indicates that a method overrides a method of a superclass
- @Deprecated indicates that the use of a class or a method is discouraged
- @SuppressWarnings turns off compiler warnings

Defining Annotation Types

```
@interface Preliminary {}          // marker annotation
@interface Copyright {
   String value();
}
@interface Enhancement {
   String synopsis();
   String assigned() default "unassigned";
   String date() default "undated";
}
```

- Annotation types are defined like interfaces
- The elements of an annotation can have default values

Using Annotations

```
@Preliminary()
@Copyright("2004 Scientific Computing")
@Enhancement(synopsis="Add complex numbers",assigned="Peter")
public class Calculator {
   ...
}
```

- Annotations are used by providing name-value pairs for each element of the corresponding type
- For single-valued annotations, the name can be omitted

Meta-Annotations

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
@interface Enhancement { ... }
```

Meta-Annotations are used to annotate annotations:

- @Target specifies which program elements can have an annotation of the defined type
- @Retention indicates whether an annotation is discarded by the compiler, kept in the class file or made available at runtime
- @Inherited indicates that the annotation type is inherited by subclasses
- @Documented indicates that the annotation should be included in the documentation

Processing Annotations using Javadoc

```
import com.sun.javadoc.*;
public class AnnotationDoclet {
   public static boolean start(RootDoc root) {
      for (ClassDoc clazz : root.classes()) {
         for (AnnotationDesc annotation : clazz.annotations())
            ...
      }
      return true;
   }
}
```

- Using the Javadoc API, the annotations of a program can be processed on the source level

Processing Annotations using Reflection

```
import java.lang.reflect.*;
public class AnnotationProcessor {
   public static void main(String args[])  throws Exception {
      Class clazz = Class.forName(args[0]);
      for (Annotation annotation : clazz.getAnnotations())
         ...
   }
}
```

- Using the Reflection API, the annotations of a program can be processed on the class-level

Formatting

Formatted Data in Pre-Tiger

```
BufferedReader in =
   new BufferedReader(new InputStreamReader(System.in));
double principal = Double.parseDouble(in.readLine());
double annualRate = Double.parseDouble(in.readLine());
int term = Integer.parseInt(in.readLine());
double payment = ...
System.out.println(new DecimalFormat("#,##0.00").format(payment));
```

- The reading and writing of formatted data is complicated

Using Scanner and Formatter

```
Scanner scanner = new Scanner(System.in);
double principal = scanner.nextDouble();
double annualRate = scanner.nextDouble();
int term = scanner.nextInt();
double payment = ...
Formatter formatter = new Formatter(System.out);
formatter.format("Monthly payment: %,5.2f%n", payment);
```

- The class java.util.Scanner can be used to convert text into primitives and strings
- The class java.util.Formatter allows for printf-style formatting of text and primitives

Format Syntax

format(String *format*, Object... *arguments*)

- *format* is the format string that contains fixed text and embedded format specifiers of the form %[*argument*][*flags*][*width*][.*precision*]*type*
- *arguments* are the objects which are converted as specified by the format specifiers in the format string

Convenience Methods

```
double payment = ...
String text = String.format("Monthly payment: %,5.2f%n", payment);
...
System.out.printf("Monthly payment: %,5.2f%n", payment);
```

- The format method of the class java.lang.String can be used to create a formatted string
- The printf method of the classes java.io.PrintStream and java.io.PrintWriter generate formatted output

Threading

Concurrency Utilities

The concurrency utilities provide low-level primitives for advanced concurrent programming:

- Task execution framework
- Locks and synchronizers
- Concurrent collections
- Atomic variables

Using Callable and Future

Callable

- represents a task that returns a result and may throw an exception

Future

- is a wrapper of an asynchronous task
- provides methods to wait for the completion of the task and to retrieve its result
- allows to cancel the task and catch its exceptions

Future Task

- creates a Runnable from a Callable and implements a Future

Callable and Future Interfaces

```
package java.util.concurrent;

public interface Callable<V> {
   public V call() throws Exception;
}
public interface Future<V> {
   public V get();
   public V get(long timeout, TimeUnit unit);
   public boolean cancel(boolean mayInterruptIfRunning);
   public boolean isCancelled();
   public boolean isDone() ;
}
public class FutureTask<V> implements Future<V>, Runnable {
   public FutureTask(Callable<V> callable);
   public void run();
   ...
}
```

Future Task Example

```
public class PrimeSearch implements Callable<Long> {
   private long min;
   public PrimeSearch(long min) { this.min = min; }
   public Long call() throws Exception {
      for (long p = min; !Thread.interrupted(); p++)
         if (isPrime(p)) return p;
      return null;
   }
   private boolean isPrime(long p) { ... }
   public static void main(String[] args) {
      PrimeSearch search = new PrimeSearch(...);
      FutureTask<Long> task = new FutureTask<Long>(search);
      new Thread(task).start();
      try {
         long prime = task.get(10, TimeUnit.SECONDS);
         ...
      }
      catch (Exception e) { task.cancel(true); }
   }
}
```


Using Executors

Executors execute submitted tasks using threads:

- the tasks are put into an internal queue
- the threads are kept in a pool

Executor implementations:

- Single thread executor
  has only one thread to run the tasks
- Fixed thread executor
  creates a fixed number of threads
- Cached thread executor
  uses as many threads as needed
- Scheduled thread executor
  allows to schedule the execution at specific times

Executor Interfaces

```
package java.util.concurrent;

public interface Excecutor {
   public void execute(Runnable task);
}
public interface ExecutorService extends Executor {
   public Future<?> submit(Runnable task);
   public boolean awaitTermination(long timeout, TimeUnit unit);
   public void shutdown();
   public List<Runnable> shutdownNow();
   ...
}
public interface ScheduledExecutorInterface extends ExecutorService {
   public ScheduledFuture<?>
      schedule(Runnable task, long delay, TimeUnit unit);
   public ScheduledFuture<?> scheduleAtFixedRate(...);
   public ScheduledFuture<?> scheduleWithFixedDelay(...);
}
```


Thread Pool Example

```
public class EchoServer {
   public static void main(String[] args) throws Exception {
      int port = Integer.parseInt(args[0]);
      ServerSocket serverSocket = new ServerSocket(port);
      ExecutorService pool = Executors.newFixedThreadPool(10);
      try {
         serverSocket.setSoTimeout(100000);
         while (true) {
            Socket socket = serverSocket.accept();
            pool.execute(new Handler(socket));
         }
      }
      catch (Exception e) { pool.shutdown(); }
}
class Handler implements Runnable {
   private Socket socket;
   Handler(Socket socket) { this.socket = socket; }
   public void run() { ... }
}
```

Scheduling Example

```
public class ReminderService {
   public static void main(String[] args) {
      ScheduledExecutorService executor =
         Executors.newSingleThreadScheduledExecutor();
      try {
         while (true) {
            ...
            Runnable reminder = new Reminder(message);
            executor.schedule(reminder, delay, TimeUnit.MILLISECONDS);
         }
      }
      catch (Exception e) { executor.shutdown(); }
   }
}
class Reminder implements Runnable {
   private String message;
   Reminder(String message) { this.message = message; }
   public void run() { ... }
}
```


Using Locks

Explicit locks

- offer different ways to acquire a lock
- are not tied to block or method boundaries
- provide platform independent wait policies

Read-write locks

- permit several readers but only one writer at the same time

Explicit conditions

- allow to associate more than one condition with a lock
- solve the nested monitor problem
- offer different wait policies

Lock and Condition Interfaces

```
package java.util.concurrent.locks ;

public interface Lock {
   public void lock();
   public boolean tryLock();
   public boolean tryLock(long time, TimeUnit unit);
   public void unlock();
   public Condition newCondition();
   ...
}
public interface Condition {
   public void await();
   public boolean await(long time, TimeUnit unit);
   public void signal();
   public void signalAll();
   ...
}
```


Locking Example

```
public class BlockingQueue<E> {
   private Lock lock = new ReentrantLock();
   private Condition notEmpty = lock.newCondition();
   private LinkedList<E> elems = new LinkedList<E>();

   public void put(E e) throws InterruptedException {
      lock.lock();
      try { elems.add(e); notEmpty.signal(); }
      finally { lock.unlock(); }
   }
   public E take() throws InterruptedException {
      lock.lock();
      try {
         while (elems.isEmpty()) notEmpty.await();
         return elems.getFirst();
      }
      finally { lock.unlock(); }
   }
}
```

Using Synchronizers

Synchronizers allow threads to wait for a specific condition:

- Semaphore
  restricts the number of threads that can access some resource
- BlockingQueue
  allows to exchange data between threads according to the producer consumer pattern
- SynchronousQueue
  allows to exchange data between threads without buffering
- Exchanger
  allows to exchange data between two threads at a common synchronization point
- CyclicBarrier
  allows a set of threads to wait for each other (rendezvous)
- CountDownLatch
  allows threads to wait for the completion of a set of operations

Concurrent Collections

Concurrent collections are optimized for concurrent use:

- ConcurrentHashMap
  segments its internal hash map such that concurrent writes are possible
- CopyOnWriteArrayList/Set
  creates a new copy of the underlying array/set for writing and then assigns it back to the original
- ConcurrentLinkedQueue
  employs an efficient wait-free algorithm to access a queue concurrently

Atomic Variables

Atomic variables offer high-performance atomic operations:

- get/set
  retrieves/sets the value
- getAndSet
  sets the value and retrieves the previous value
- compareAndSet
  checks the value and if it matches, sets it to a new value
- increment/decrementAndGet
  increments/decrements the value and returns the new value
- getAndIncrement/Decrement
  returns the current value and increments/decrements it in the variable